

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

```
In [2]: df=pd.read_csv('placementdata.csv')
df.head()
```

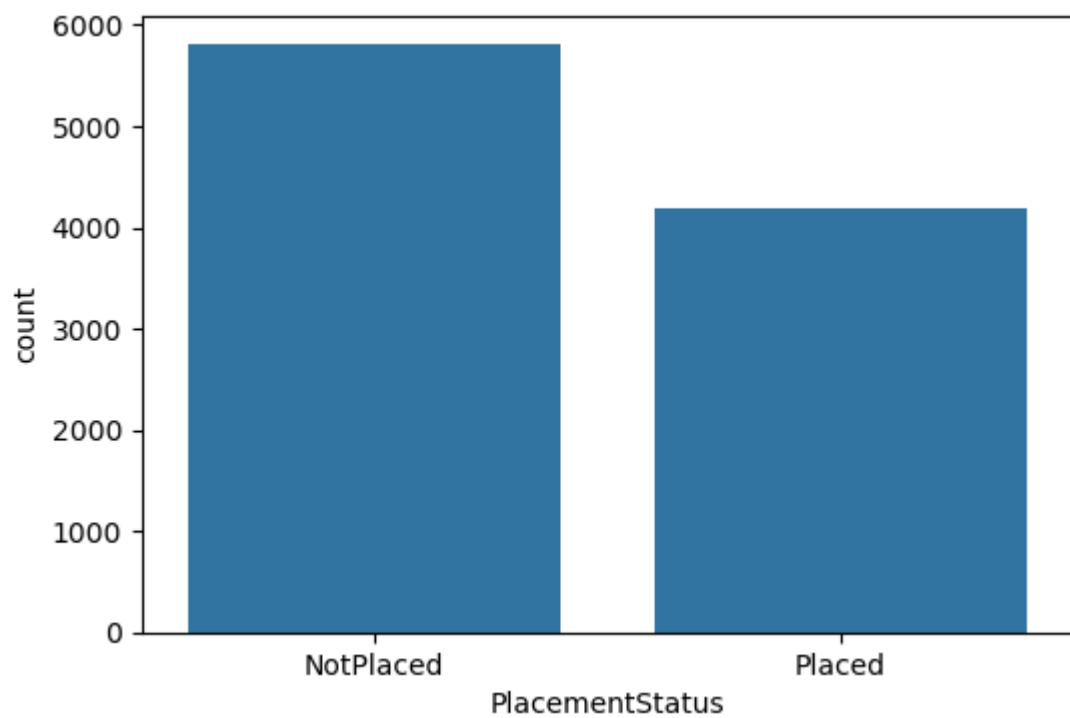
```
Out[2]:
```

	StudentID	CGPA	Internships	Projects	Workshops/Certifications	AptitudeTestScore
0	1	7.5	1	1		1
1	2	8.9	0	3		2
2	3	7.3	1	2		2
3	4	7.5	1	1		2
4	5	8.3	1	2		2

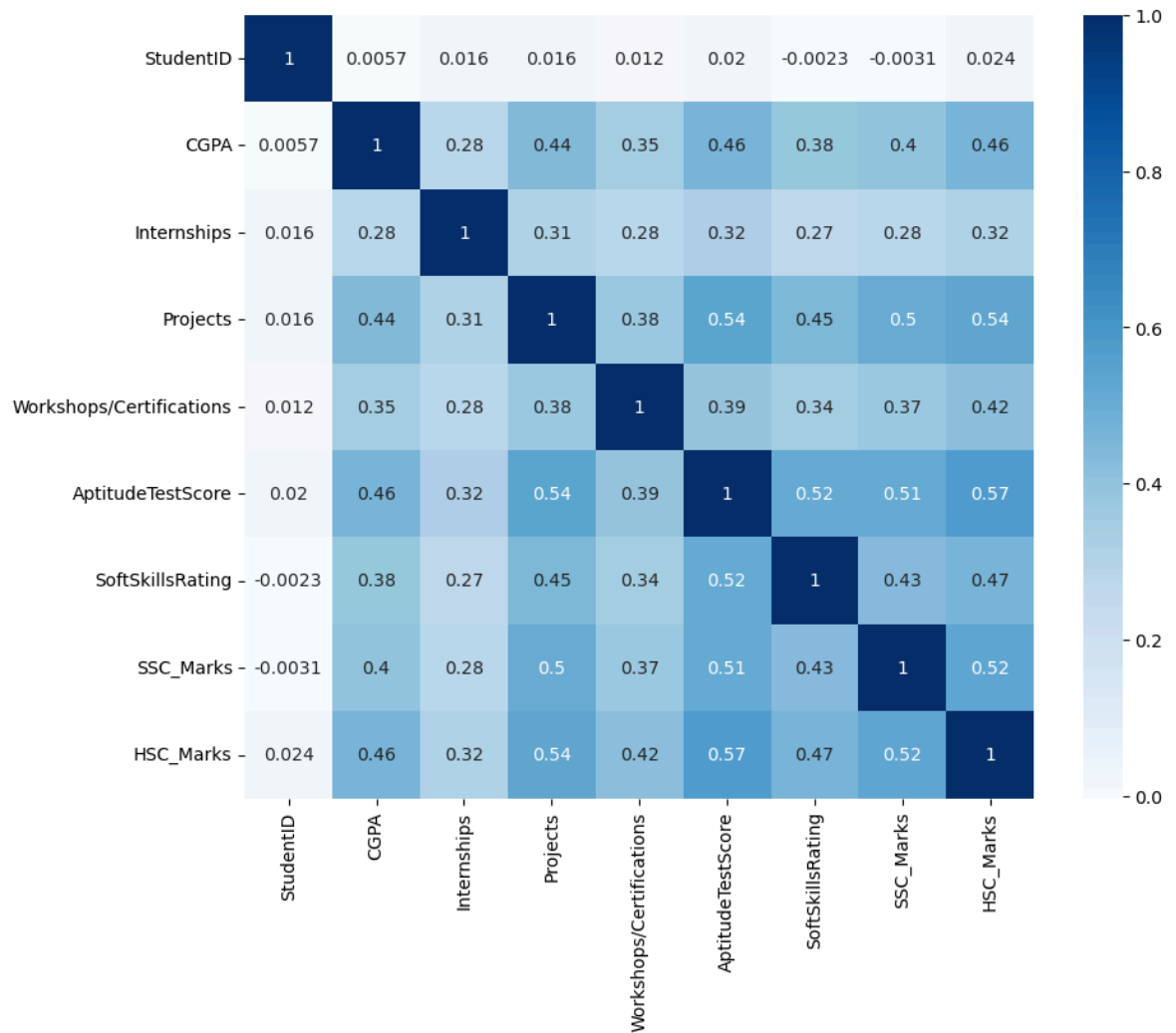
```
In [3]: print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
```

```
(10000, 12)
StudentID          int64
CGPA               float64
Internships        int64
Projects           int64
Workshops/Certifications int64
AptitudeTestScore int64
SoftSkillsRating   float64
ExtracurricularActivities object
PlacementTraining  object
SSC_Marks          int64
HSC_Marks          int64
PlacementStatus    object
dtype: object
StudentID          0
CGPA               0
Internships        0
Projects           0
Workshops/Certifications 0
AptitudeTestScore 0
SoftSkillsRating   0
ExtracurricularActivities 0
PlacementTraining  0
SSC_Marks          0
HSC_Marks          0
PlacementStatus    0
dtype: int64
```

```
In [4]: plt.figure(figsize=(6,4))
sns.countplot(x='PlacementStatus',data=df)
plt.show()
```



```
In [5]: numeric_df=df.select_dtypes(include=np.number)
plt.figure(figsize=(10,8))
sns.heatmap(numeric_df.corr(),annot=True,cmap='Blues')
plt.show()
```



```
In [6]: le=LabelEncoder()
for c in ['PlacementStatus','PlacementTraining','ExtracurricularActivities']:
    df[c]=le.fit_transform(df[c])

X=df.drop(['StudentID','PlacementStatus'],axis=1)
y=df['PlacementStatus']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_s
```

```
In [7]: dt=DecisionTreeClassifier(random_state=42)
dt.fit(X_train,y_train)
dt_pred=dt.predict(X_test)

rf=RandomForestClassifier(n_estimators=100,random_state=42)
rf.fit(X_train,y_train)
rf_pred=rf.predict(X_test)
```

```
In [8]: def evaluate(name,y_true,y_pred):
    acc=accuracy_score(y_true,y_pred)
    pre=precision_score(y_true,y_pred)
    rec=recall_score(y_true,y_pred)
    f1=f1_score(y_true,y_pred)
    print(name)
    print('Accuracy:',round(acc*100,2))
    print(classification_report(y_true,y_pred))
    cm=confusion_matrix(y_true,y_pred)
    sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
    plt.title(name)
```

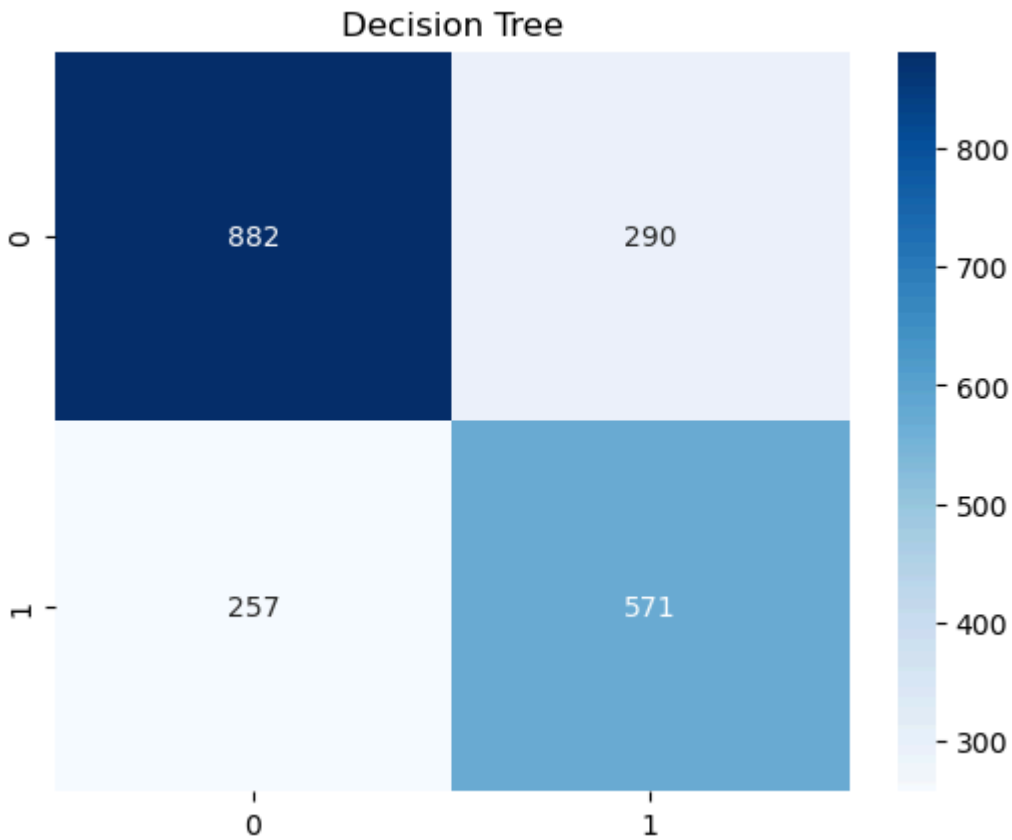
```
plt.show()
return [acc,pre,rec,f1]

dtm=evaluate('Decision Tree',y_test,dt_pred)
rfm=evaluate('Random Forest',y_test,rf_pred)
```

Decision Tree

Accuracy: 72.65

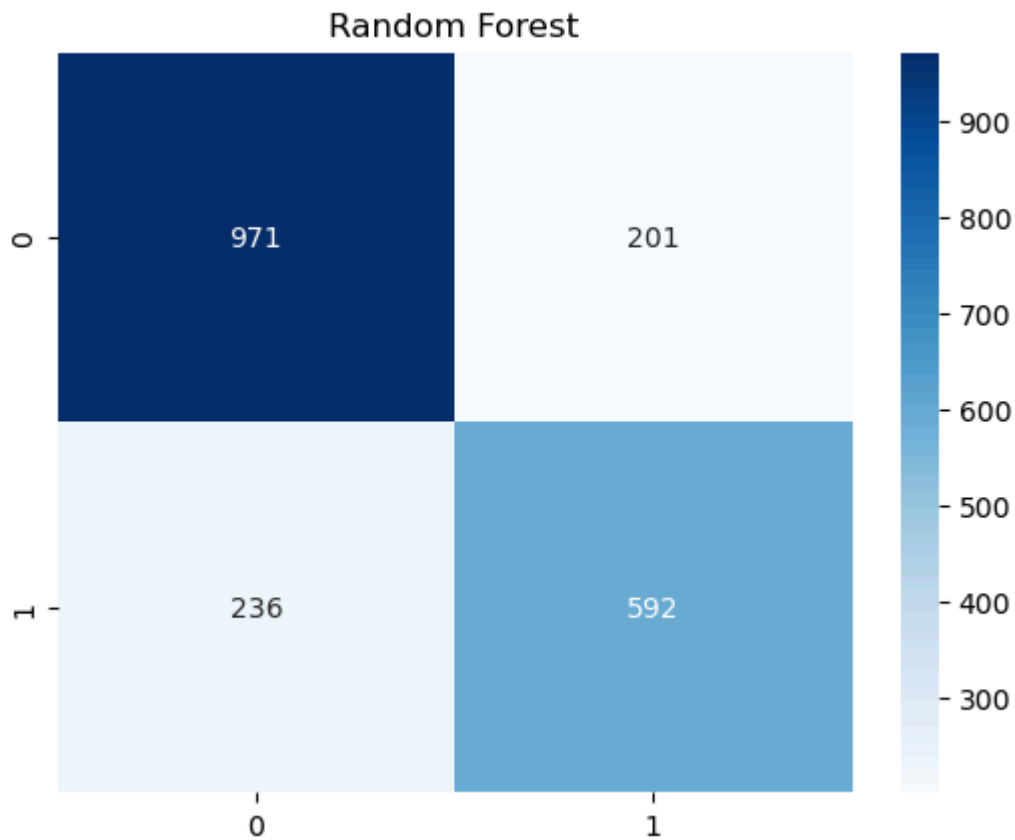
	precision	recall	f1-score	support
0	0.77	0.75	0.76	1172
1	0.66	0.69	0.68	828
accuracy			0.73	2000
macro avg	0.72	0.72	0.72	2000
weighted avg	0.73	0.73	0.73	2000



Random Forest

Accuracy: 78.15

	precision	recall	f1-score	support
0	0.80	0.83	0.82	1172
1	0.75	0.71	0.73	828
accuracy			0.78	2000
macro avg	0.78	0.77	0.77	2000
weighted avg	0.78	0.78	0.78	2000



```
In [9]: comparison=pd.DataFrame({
'Model':['Decision Tree','Random Forest'],
'Accuracy (%)':[dtm[0]*100,rfm[0]*100],
'Precision':[dtm[1],rfm[1]],
'Recall':[dtm[2],rfm[2]],
'F1 Score':[dtm[3],rfm[3]]
})
print(comparison)
sns.barplot(x='Model',y='Accuracy (%)',data=comparison)
plt.show()
```

	Model	Accuracy (%)	Precision	Recall	F1 Score
0	Decision Tree	72.65	0.663182	0.689614	0.676140
1	Random Forest	78.15	0.746532	0.714976	0.730413

